

Technical Documentation: Integration of Claude MCP and Kali Linux

1. Architectural Overview

The experimental setup utilizes a distributed architecture to bridge a high-level Large Language Model (LLM) with an isolated offensive security environment. The system consists of three main layers:

- **The Interface (Client):** Claude Desktop running on macOS, serving as the user interface and the reasoning engine.
- **The Transport Layer:** Secure Shell (SSH) tunneling Model Context Protocol (MCP) packets between the host and the guest.
- **The Operational Environment (Target Node):** A Kali Linux Virtual Machine (VM) hosting a specialized MCP API server that interfaces with native security tools.
- **The Target:** Damn Vulnerable Web Application (DVWA) deployed via Docker, accessible over the Tailscale mesh network.

2. Kali Linux Server-Side Configuration

To enable the LLM to execute system-level commands, the official `mcp-kali-server` package was implemented. This acts as a bridge between the model's requests and the OS-level binaries.

Core installation: `sudo apt update && sudo apt install -y mcp-kali-server`

Dependency Resolution (The "Pydantic Fix"): A significant technical challenge was encountered due to a version mismatch between the Kali MCP package and the system-wide `Pydantic v2.10` library. This was resolved by implementing a Python **Virtual Environment (venv)** to isolate and downgrade critical dependencies:

1. **Environment Isolation:** Created a dedicated venv via `python3 -m venv ~/kali-mcp-fix/venv`.
2. **Version Constraint:** Forced the installation of compatible library versions:
 - `pydantic<2.10`
 - `flask`, `flask-restful`, `requests`, and `mcp`.

Commands performed:

1. Create a directory for the fix and initialize the virtual environment

```
mkdir -p ~/kali-mcp-fix
```

```
cd ~/kali-mcp-fix
```

```
python3 -m venv venv
```

2. Activate the virtual environment

```
source venv/bin/activate
```

3. Install compatible versions of all required libraries

Note: Quotes are used to prevent the shell from misinterpreting the "<" symbol

```
pip install "pydantic<2.10" flask flask-restful requests mcp
```

Start a new terminal or session, activate venv, and run the client to verify functioning bridge

```
source ~/kali-mcp-fix/venv/bin/activate
```

```
python3 /usr/share/mcp-kali-server/mcp_server.py
```

3. MCP Bridge Middleware

The communication is handled by two distinct Python processes running within the Kali VM:

- **The API Engine (`kali_server.py`):** A Flask-based REST API that listens on `localhost:5000`, providing an endpoint for security tool execution.
- **The MCP Client (`mcp_server.py`):** The primary entry point for Claude. It translates MCP-standardized JSON-RPC calls into API requests for the engine.

Commands performed:

Ensure venv is active

```
source ~/kali-mcp-fix/venv/bin/activate
```

Start the Kali Tools API server in the background

```
python3 /usr/share/mcp-kali-server/kali_server.py --ip 127.0.0.1 --port 5000
```

Redirecting output to nohup ensures it keeps running if the terminal closes

```
nohup python3 /usr/share/mcp-kali-server/kali_server.py --ip 127.0.0.1 --port 5000 > mcp.log  
2>&1 &
```

4. Claude Desktop Integration (macOS)

The integration was finalized by configuring the `claude_desktop_config.json` file. This configuration instructs the LLM to initiate an SSH session to the Kali VM and execute the MCP client using the Python interpreter located within the isolated virtual environment.

Config snippet:

```
{  
  
  "mcpServers": {  
  
    "kali-master": {  
  
      "command": "ssh",  
  
      "args": [  
  
        "kali@<KALI_IP_ADDRESS>",  
  
        "/home/kali/kali-mcp-fix/venv/bin/python3",  
  
        "/usr/share/mcp-kali-server/mcp_server.py"  
  
      ]  
  
    }  
  
  }  
  
}
```

5. Verification and Operational Status

Success was verified through a "Synthetic Handshake," where the model autonomously queried the environment for available binaries.

- **Verification Command:** `nmap --version` executed via Claude's tool-calling interface.
- **Health Status:** Confirmed as `healthy` with direct access to the full Kali suite (`nmap`, `nikto`, `sqlmap`, `dirb`, etc.).